```
from sklearn.datasets import load_breast_cancer
import pandas as pd
# Load the dataset
breast_cancer = load_breast_cancer()
# Create a DataFrame
df_cancer = pd.DataFrame(data=breast_cancer.data,
columns=breast_cancer.feature_names)
# Add the target variable
df_cancer['target'] = breast_cancer.target
# Display the first few rows
df_cancer.head()
```

{"type":"dataframe","variable_name":"df_cancer"}

```
#a) Load the dataset and display the first five rows

import pandas as pd
from sklearn.datasets import load_breast_cancer

# Load the dataset
cancer = load_breast_cancer()

# Convert to a Pandas DataFrame
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

# Add the diagnosis column
df["diagnosis"] = pd.Series(cancer.target).map({
    0: "malignant",
    1: "benign"
})

# Display the first five rows
print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
0        17.99         10.38          122.80     1001.0
0.11840
1        20.57         17.77          132.90     1326.0
0.08474
2        19.69         21.25          130.00     1203.0
0.10960
3        11.42         20.38           77.58      386.1
0.14250
4        20.29         14.34          135.10     1297.0
0.10030

   mean compactness  mean concavity  mean concave points  mean
symmetry  \
0           0.27760          0.3001              0.14710
```

```
0.2419
1           0.07864          0.0869               0.07017
0.1812
2           0.15990          0.1974               0.12790
0.2069
3           0.28390          0.2414               0.10520
0.2597
4           0.13280          0.1980               0.10430
0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst
area  \
0                 0.07871  ...          17.33           184.60
2019.0
1                 0.05667  ...          23.41           158.80
1956.0
2                 0.05999  ...          25.53           152.50
1709.0
3                 0.09744  ...          26.50            98.87
567.7
4                 0.05883  ...          16.67           152.20
1575.0

   worst smoothness  worst compactness  worst concavity  worst concave
points  \
0            0.1622             0.6656           0.7119
0.2654
1            0.1238             0.1866           0.2416
0.1860
2            0.1444             0.4245           0.4504
0.2430
3            0.2098             0.8663           0.6869
0.2575
4            0.1374             0.2050           0.4000
0.1625

   worst symmetry  worst fractal dimension  diagnosis
0          0.4601                  0.11890  malignant
1          0.2750                  0.08902  malignant
2          0.3613                  0.08758  malignant
3          0.6638                  0.17300  malignant
4          0.2364                  0.07678  malignant

[5 rows x 31 columns]
```

```
# b). What is the shape of the dataset? How many rows and columns does
it have?
print(df.columns)
print("Dataset shape:", df.shape)
```

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal
dimension',
       'radius error', 'texture error', 'perimeter error', 'area
error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error', 'fractal dimension
error',
       'worst radius', 'worst texture', 'worst perimeter', 'worst
area',
       'worst smoothness', 'worst compactness', 'worst concavity',
       'worst concave points', 'worst symmetry', 'worst fractal
dimension',
       'diagnosis'],
      dtype='object')
Dataset shape: (569, 31)
Number of rows: 569
Number of columns: 31
```

```
# c). Use the `info()` function to display the data types of each
column. Are all columns of the appropriate data type for analysis?
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   mean radius             569 non-null    float64
 1   mean texture            569 non-null    float64
 2   mean perimeter          569 non-null    float64
 3   mean area               569 non-null    float64
 4   mean smoothness         569 non-null    float64
 5   mean compactness        569 non-null    float64
 6   mean concavity          569 non-null    float64
 7   mean concave points     569 non-null    float64
 8   mean symmetry           569 non-null    float64
 9   mean fractal dimension  569 non-null    float64
 10  radius error            569 non-null    float64
 11  texture error           569 non-null    float64
 12  perimeter error         569 non-null    float64
 13  area error              569 non-null    float64
 14  smoothness error        569 non-null    float64
 15  compactness error       569 non-null    float64
 16  concavity error         569 non-null    float64
 17  concave points error    569 non-null    float64
```

```
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
 30  diagnosis                569 non-null    object
dtypes: float64(30), object(1)
memory usage: 137.9+ KB
```

```
# d). Use the `describe()` function to display summary statistics for
the numerical columns.What are the mean, median, and standard
deviation of `mean radius`?
print(df.describe())
```

```
       mean radius  mean texture  mean perimeter    mean area  \
count   569.000000    569.000000      569.000000   569.000000
mean     14.127292     19.289649       91.969033   654.889104
std       3.524049      4.301036       24.298981   351.914129
min       6.981000      9.710000       43.790000   143.500000
25%      11.700000     16.170000       75.170000   420.300000
50%      13.370000     18.840000       86.240000   551.100000
75%      15.780000     21.800000      104.100000   782.700000
max      28.110000     39.280000      188.500000  2501.000000

       mean smoothness  mean compactness  mean concavity  mean concave
points  \
count       569.000000        569.000000      569.000000
569.000000
mean          0.096360          0.104341        0.088799
0.048919
std           0.014064          0.052813        0.079720
0.038803
min           0.052630          0.019380        0.000000
0.000000
25%           0.086370          0.064920        0.029560
0.020310
50%           0.095870          0.092630        0.061540
0.033500
75%           0.105300          0.130400        0.130700
0.074000
max           0.163400          0.345400        0.426800
0.201200
```

```
       mean symmetry  mean fractal dimension  ...  worst radius  \
count     569.000000              569.000000  ...    569.000000
mean        0.181162                0.062798  ...     16.269190
std         0.027414                0.007060  ...      4.833242
min         0.106000                0.049960  ...      7.930000
25%         0.161900                0.057700  ...     13.010000
50%         0.179200                0.061540  ...     14.970000
75%         0.195700                0.066120  ...     18.790000
max         0.304000                0.097440  ...     36.040000

       worst texture  worst perimeter   worst area  worst
smoothness  \
count     569.000000       569.000000   569.000000        569.000000

mean       25.677223       107.261213   880.583128          0.132369

std         6.146258        33.602542   569.356993          0.022832

min        12.020000        50.410000   185.200000          0.071170

25%        21.080000        84.110000   515.300000          0.116600

50%        25.410000        97.660000   686.500000          0.131300

75%        29.720000       125.400000  1084.000000          0.146000

max        49.540000       251.200000  4254.000000          0.222600


       worst compactness  worst concavity  worst concave points  \
count         569.000000       569.000000            569.000000
mean            0.254265         0.272188              0.114606
std             0.157336         0.208624              0.065732
min             0.027290         0.000000              0.000000
25%             0.147200         0.114500              0.064930
50%             0.211900         0.226700              0.099930
75%             0.339100         0.382900              0.161400
max             1.058000         1.252000              0.291000

       worst symmetry  worst fractal dimension
count      569.000000               569.000000
mean         0.290076                 0.083946
std          0.061867                 0.018061
min          0.156500                 0.055040
25%          0.250400                 0.071460
50%          0.282200                 0.080040
75%          0.317900                 0.092080
max          0.663800                 0.207500

[8 rows x 30 columns]
```

```python
# e). Are there any missing values in the dataset? If so, which
columns contain them, and how many missing values are there?
# Count missing values in each column
print(df.isnull().sum())

# Total missing values
print("Total missing values:", df.isnull().sum().sum())
missing = df.isnull().sum()

print(missing[missing > 0])
```

```
mean radius                  0
mean texture                 0
mean perimeter               0
mean area                    0
mean smoothness              0
mean compactness             0
mean concavity               0
mean concave points          0
mean symmetry                0
mean fractal dimension       0
radius error                 0
texture error                0
perimeter error              0
area error                   0
smoothness error             0
compactness error            0
concavity error              0
concave points error         0
symmetry error               0
fractal dimension error      0
worst radius                 0
worst texture                0
worst perimeter              0
worst area                   0
worst smoothness             0
worst compactness            0
worst concavity              0
worst concave points         0
worst symmetry               0
worst fractal dimension      0
diagnosis                    0
dtype: int64
Total missing values: 0
Series([], dtype: int64)
```

```python
#2. Data Cleaning
#a). If there are any missing values, decide whether to remove them or
fill them with appropriate values. What approach did you choose, and
why?
```

```python
# Check missing values in each column
print(df.isnull().sum())

# Calculate total missing values
print("Total missing values:", df.isnull().sum().sum())

# Fill missing numerical values with the column median
numeric_cols = df.select_dtypes(include="number").columns

df[numeric_cols] = df[numeric_cols].fillna(
    df[numeric_cols].median()
)

# Remove rows with missing values in non-numerical columns
df = df.dropna()

print("Missing values after cleaning:", df.isnull().sum().sum())


'''
Answer and explanation:

The original Breast Cancer Wisconsin Diagnostic dataset contains no
missing values.
Therefore, no missing-value treatment is necessary for the original
dataset.
If missing values were present in another version, I would consider
filling numerical
features with their median because it is less affected by extreme
values than the mean. Rows with missing categorical values could be
handled
based on the extent and cause of missingness rather than automatically
removed.
'''
```

```
mean radius                0
mean texture               0
mean perimeter             0
mean area                  0
mean smoothness            0
mean compactness           0
mean concavity             0
mean concave points        0
mean symmetry              0
mean fractal dimension     0
radius error               0
texture error              0
perimeter error            0
area error                 0
```

```
smoothness error            0
compactness error           0
concavity error             0
concave points error        0
symmetry error              0
fractal dimension error     0
worst radius                0
worst texture               0
worst perimeter             0
worst area                  0
worst smoothness            0
worst compactness           0
worst concavity             0
worst concave points        0
worst symmetry              0
worst fractal dimension     0
diagnosis                   0
dtype: int64
Total missing values: 0
Missing values after cleaning: 0
```

```
#b). Ensure that the target variable (`target`) is correctly labeled (0 for malignant, 1 for benign). Are there any inconsistencies in the target variable?
from sklearn.datasets import load_breast_cancer
import pandas as pd

cancer = load_breast_cancer()

# Create a DataFrame with the original target values
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df["target"] = cancer.target

# Check target values and their counts
print("Unique target values:", df["target"].unique())
print("\nTarget counts:")
print(df["target"].value_counts().sort_index())

# Check for invalid target values
print("Invalid target values:",
      df.loc[~df["target"].isin([0, 1]), "target"].unique())

# Convert target values to readable labels
df["diagnosis"] = df["target"].map({
    0: "Malignant",
    1: "Benign"
})

print(df[["target", "diagnosis"]].head())
```

```
'''
Are there any inconsistencies?
No. The original dataset uses 0 for malignant and 1 for benign, with
569 total samples. There are no invalid target values in the original
dataset.

Conclusion: The dataset requires no missing-value treatment, and the
target variable is correctly labeled. Always verify the target
encoding before
analysis, especially if you are working with a different CSV file.
'''

Unique target values: [0 1]

Target counts:
target
0    212
1    357
Name: count, dtype: int64
Invalid target values: []
   target  diagnosis
0       0  Malignant
1       0  Malignant
2       0  Malignant
3       0  Malignant
4       0  Malignant

import pandas as pd
from sklearn.datasets import load_breast_cancer

# Load the Breast Cancer dataset
cancer = load_breast_cancer()

# Create the DataFrame
df = pd.DataFrame(
    cancer.data,
    columns=cancer.feature_names
)

# Add the target column (0 = malignant, 1 = benign)
df["target"] = cancer.target

# Calculate descriptive statistics
features = ["mean area", "mean compactness"]

for col in features:
    print("\nFeature:", col)
    print("Mean:", df[col].mean())
```

```
    print("Median:", df[col].median())
    print("Standard Deviation:", df[col].std())


Feature: mean area
Mean: 654.8891036906855
Median: 551.1
Standard Deviation: 351.9141291816527

Feature: mean compactness
Mean: 0.10434098418277679
Median: 0.09263
Standard Deviation: 0.0528127579325122

'''
3. Descriptive Statistics – Breast Cancer Dataset
a)Calculate and interpret the mean, median, and standard deviation for
the `mean area` and `mean compactness`.
What do these statistics tell you about the data?
'''
# Select the features
features = ["mean area", "mean compactness"]

# Calculate statistics
for col in features:
    print("\nFeature:", col)
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
    print("Standard Deviation:", df[col].std())


Feature: mean area
Mean: 654.8891036906855
Median: 551.1
Standard Deviation: 351.9141291816527

Feature: mean compactness
Mean: 0.10434098418277679
Median: 0.09263
Standard Deviation: 0.0528127579325122

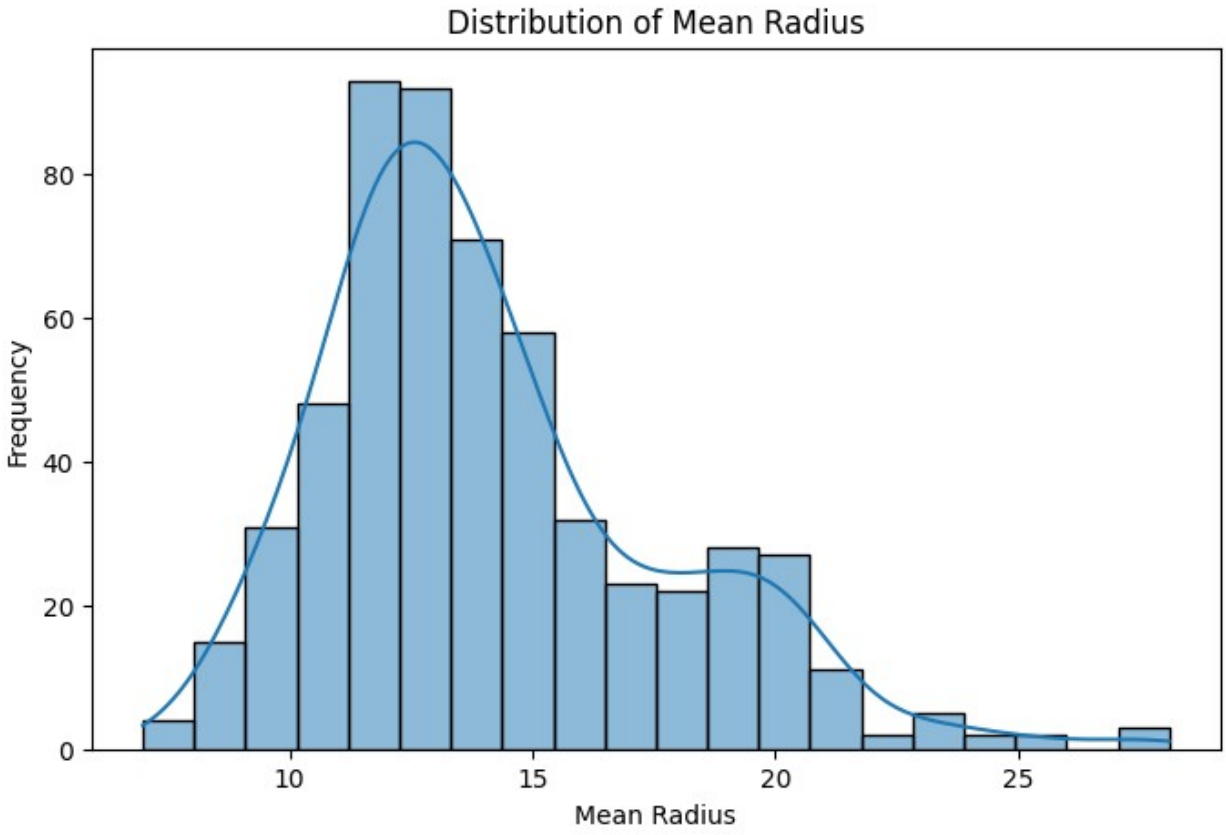
'''
b)Create a histogram for mean radius. What does the distribution look
like? Is it skewed?
'''
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))

sns.histplot(data=df, x="mean radius", bins=20, kde=True)
```

```
plt.title("Distribution of Mean Radius")
plt.xlabel("Mean Radius")
plt.ylabel("Frequency")
plt.show()

'''
Interpretation:
The distribution of mean radius is generally right-skewed (positively
skewed): most observations occur at lower-to-middle radius values,
with a tail extending toward larger values.
The histogram helps to see this pattern. The exact appearance may vary
slightly with the number of bins.
```




```
'''
c).Use a boxplot to visualize the distribution of `mean texture`. Are
there any outliers in the
data? What is the interquartile range (IQR)?
'''
plt.figure(figsize=(8, 5))

sns.boxplot(x=df["mean texture"])
```

```
plt.title("Boxplot of Mean Texture")
plt.xlabel("Mean Texture")
plt.show()

# Calculate quartiles and IQR
Q1 = df["mean texture"].quantile(0.25)
Q3 = df["mean texture"].quantile(0.75)
IQR = Q3 - Q1

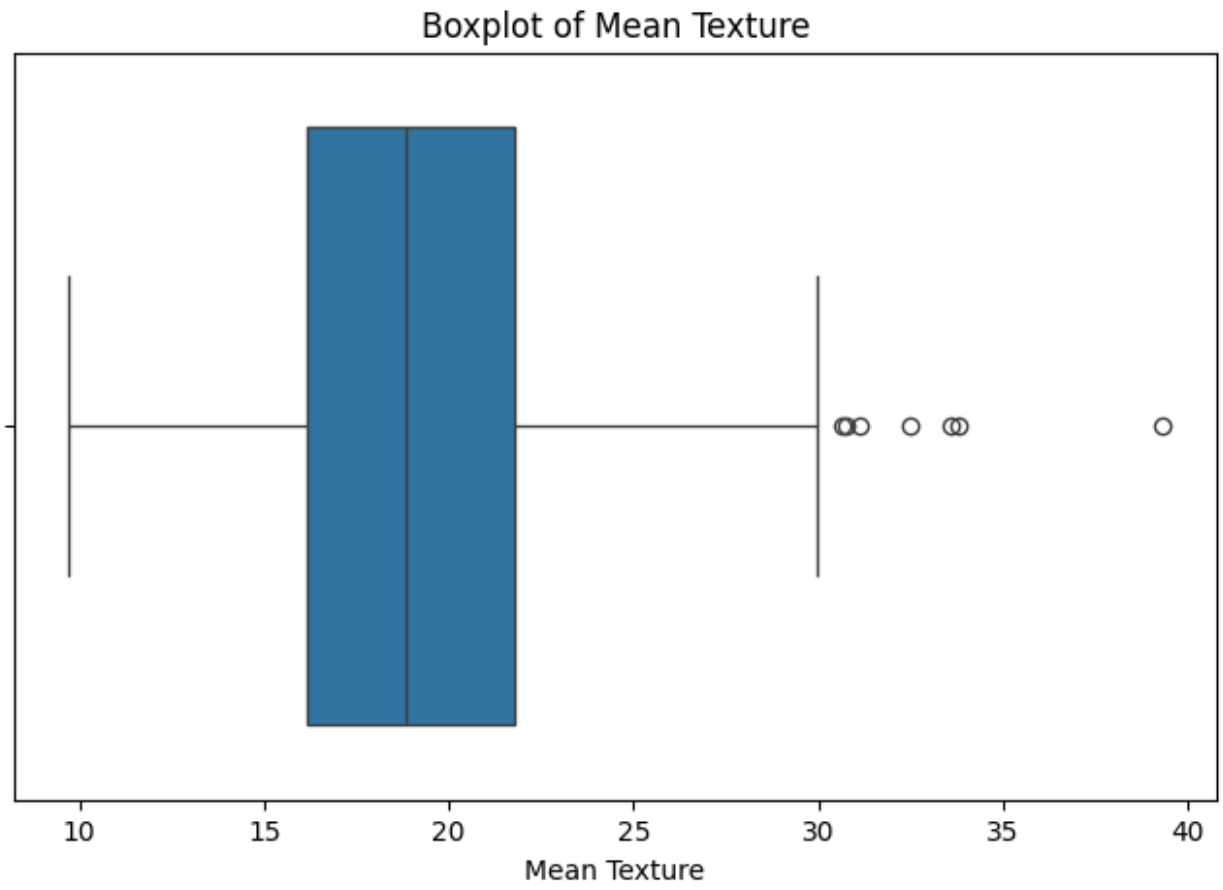
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)

# Identify potential outliers using the 1.5 × IQR rule
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

outliers = df[
    (df["mean texture"] < lower_limit) |
    (df["mean texture"] > upper_limit)
]

print("Number of potential outliers:", len(outliers))

'''
```




```
Q1: 16.17
Q3: 21.8
IQR: 5.629999999999999
Number of potential outliers: 7
```

```
'''
d). Group the data by the `target` variable and calculate the mean for
`mean perimeter`. Is
there a noticeable difference between the mean perimeter of benign and
malignant cases?
'''
# Calculate average mean perimeter for each target
perimeter_mean = df.groupby("target")["mean perimeter"].mean()

print(perimeter_mean)
print("Mean perimeter by diagnosis:")

print(
    df.groupby("target")["mean perimeter"]
    .mean()
    .rename(index={0: "Malignant", 1: "Benign"})
)
```

```
'''
Interpretation
Yes, there is a noticeable difference between the two groups. The
average mean perimeter is
substantially higher for malignant cases than for benign cases in this
dataset.

This suggests that mean perimeter may be a useful feature for
distinguishing the two groups.
However, this difference alone does not establish a diagnosis or prove
that perimeter causes malignancy.

'''
```

```
target
0    115.365377
1     78.075406
Name: mean perimeter, dtype: float64
Mean perimeter by diagnosis:
target
Malignant    115.365377
Benign        78.075406
Name: mean perimeter, dtype: float64
```

```
'''
4. Data Visualization – Breast Cancer Dataset
a). Create a bar chart to show the number of benign vs. malignant
cases. What does the chart
tell you about the distribution of cases?
'''
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Count the cases
target_counts = df["target"].value_counts().sort_index()

# Create the bar chart
plt.figure(figsize=(7, 5))

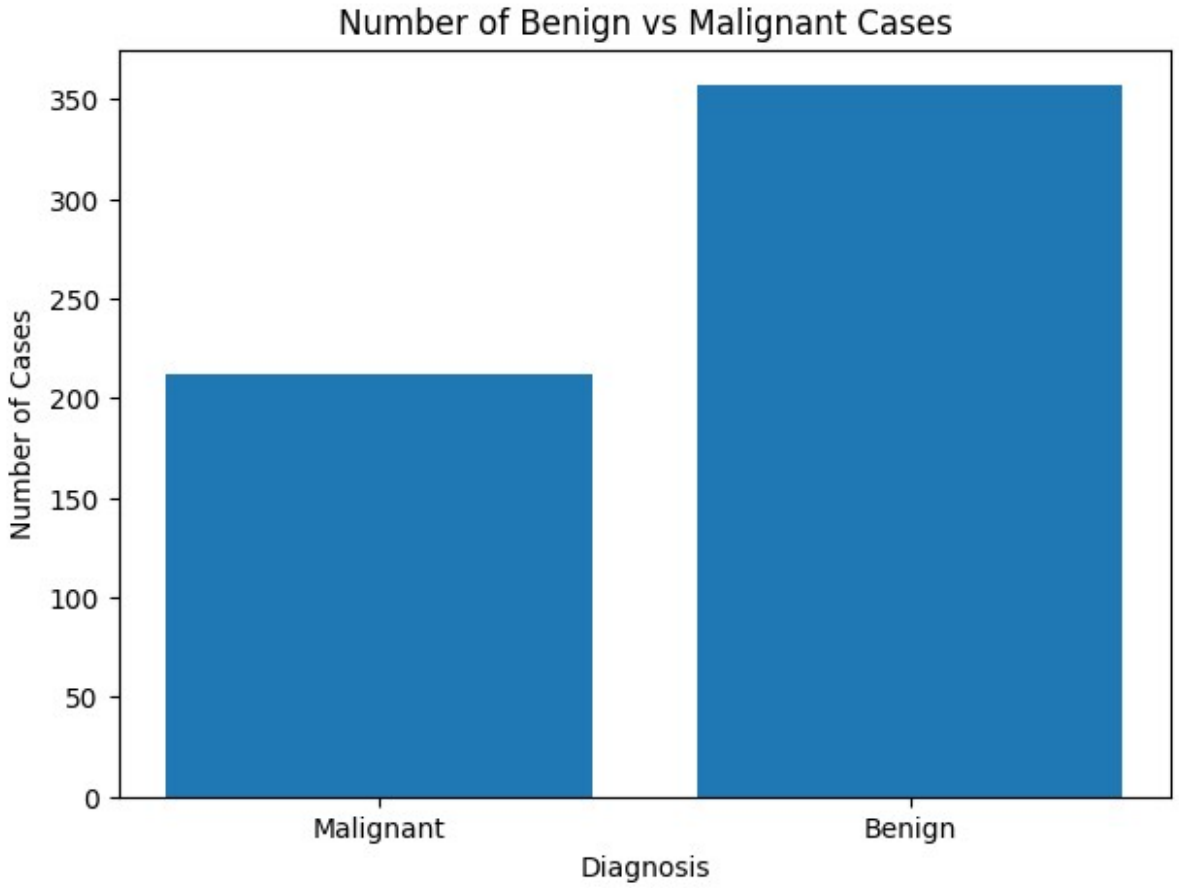
plt.bar(
    ["Malignant", "Benign"],
    target_counts.values
)

plt.title("Number of Benign vs Malignant Cases")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Cases")
```

```
plt.show()

print(target_counts.rename(index={
    0: "Malignant",
    1: "Benign"
}))

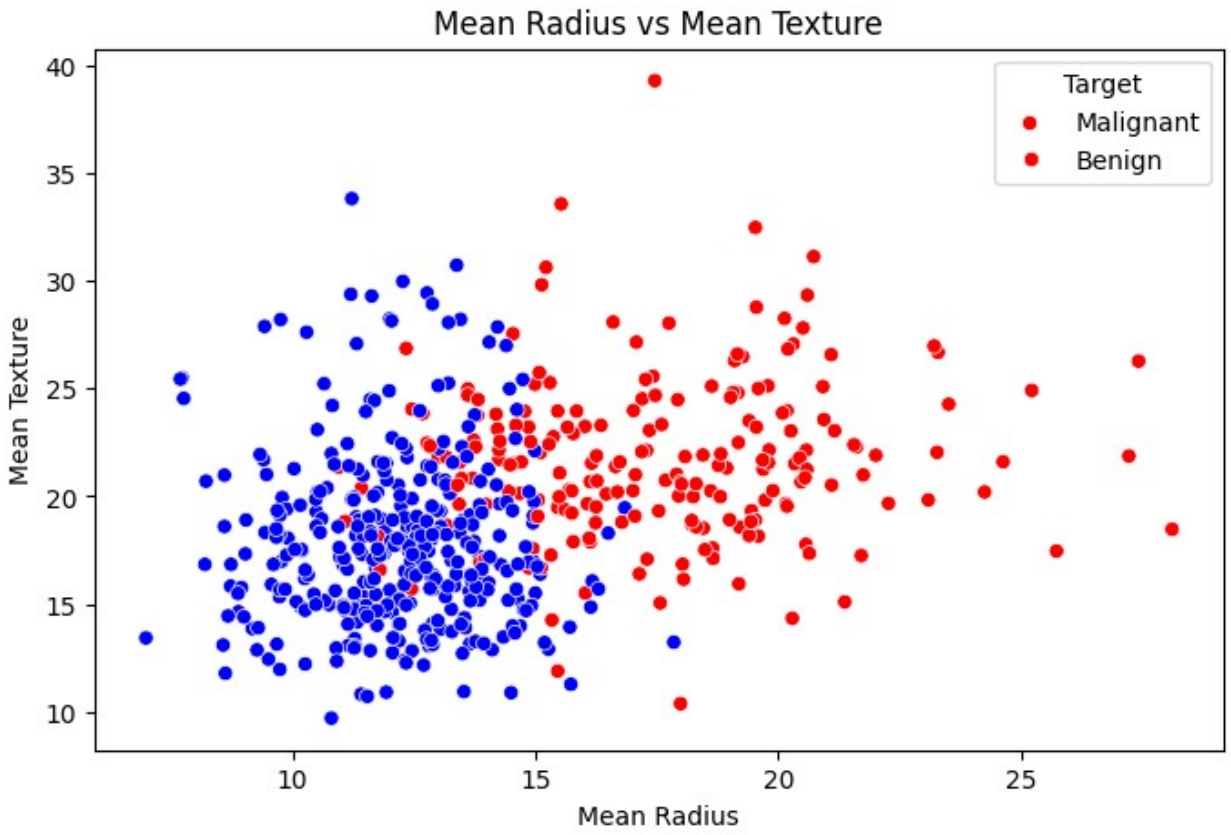
'''
Interpretation: The chart shows that benign cases are more common than
malignant cases. Approximately
62.7% of the samples are benign and 37.3% are malignant. Therefore,
the dataset has an unequal distribution
of the two classes.
'''
```




```
target
Malignant    212
Benign       357
Name: count, dtype: int64
 '''
 b). Visualize the relationship between mean radius and mean texture
```

```
using a scatter plot.
 '''
 plt.figure(figsize=(8, 5))
 sns.scatterplot(
    data=df,
    x="mean radius",
    y="mean texture",
    hue="target",
    palette={0: "red", 1: "blue"}
)
 plt.title("Mean Radius vs Mean Texture")
 plt.xlabel("Mean Radius")
 plt.ylabel("Mean Texture")
 plt.legend(title="Target", labels=["Malignant", "Benign"])
 plt.show()
 correlation = df["mean radius"].corr(df["mean texture"])
 print("Correlation:", correlation)

'''
Interpretation: The scatter plot shows how radius and texture vary
together. In this dataset, their linear
correlation is weak and positive, approximately 0.32. This means
larger mean radius values tend to be associated
with somewhat higher mean texture values, but the relationship is not
strong.
```




```
Correlation: 0.323781890927733

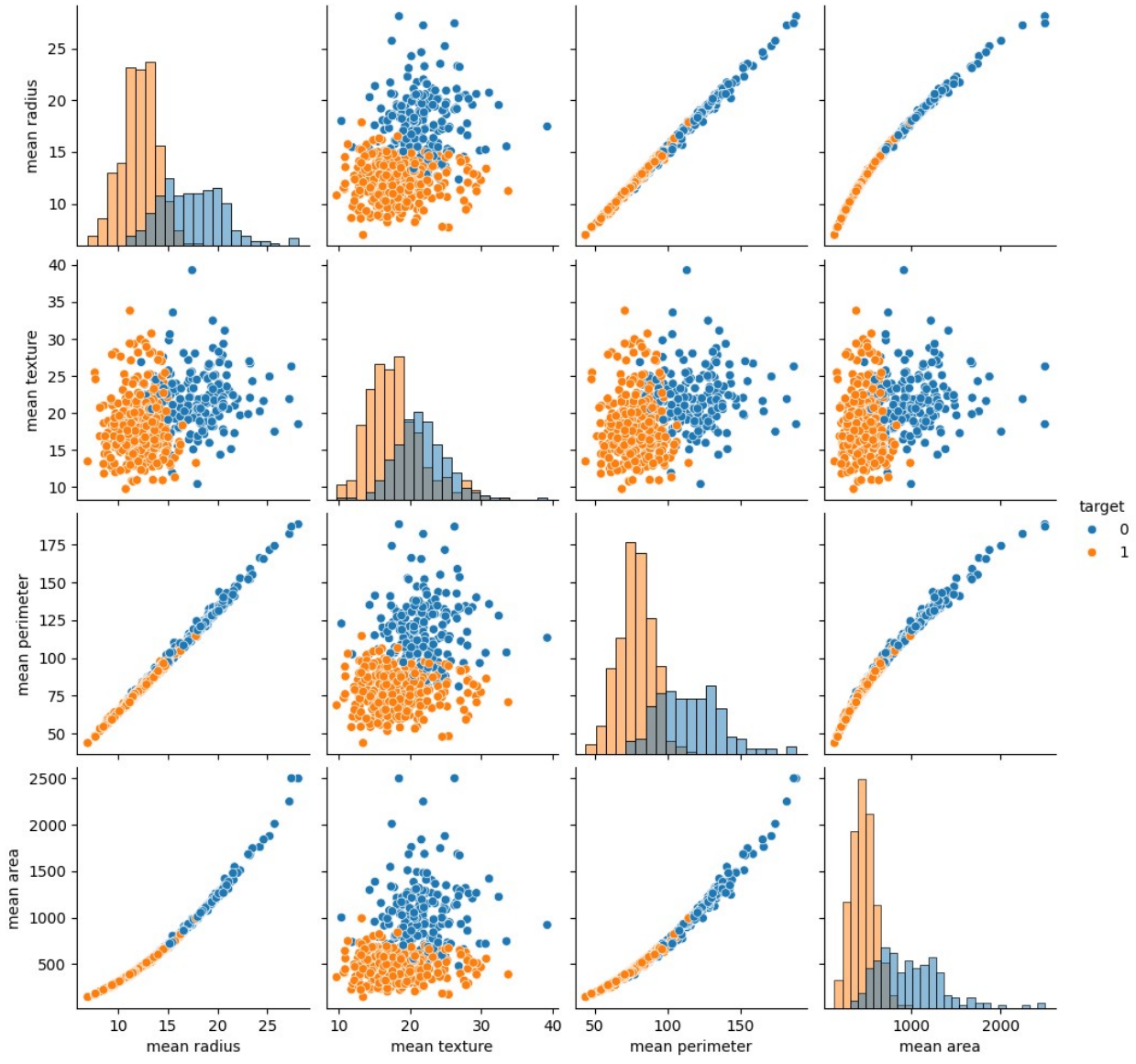
'''
c) Use a pairplot to visualize the relationship between all features
and the `target` variable.
What relationships or clusters can you identify from the pairplot?
'''
# Create readable diagnosis labels
df["diagnosis"] = df["target"].map({
    0: "Malignant",
    1: "Benign"
})

# Select representative features
selected_features = [
    "mean radius",
    "mean texture",
    "mean perimeter",
    "mean area",
    "target"
]

sns.pairplot(
    df[selected_features],
```

```
    hue="target",
    diag_kind="hist"
)

plt.show()
```




```
'''
d). Create a heatmap to visualize the correlation matrix of the
dataset.
Which features are most strongly correlated with each other? How might
this
affect your analysis?

Interpretation:
```

```
The strongest positive correlations are generally between measurements
of similar size, especially:
- mean radius and mean perimeter
- mean radius and mean area
- mean perimeter and mean area

These relationships make sense because radius, perimeter, and area all
describe the size of
cell nuclei.
How does this affect the analysis?
--Highly correlated features may contain overlapping information.
--In predictive modelling, strong correlations can cause
multicollinearity
  in some models, such as linear regression.
--Correlation does not imply causation.
--Features should not be removed automatically; their usefulness
depends
  on the analysis objective and model.

'''
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Select numerical features only
numeric_df = df.select_dtypes(include="number")

# Calculate the correlation matrix
corr_matrix = numeric_df.corr()

# Plot the heatmap
plt.figure(figsize=(16, 12))

sns.heatmap(
    corr_matrix,
    cmap="coolwarm",
    center=0
)

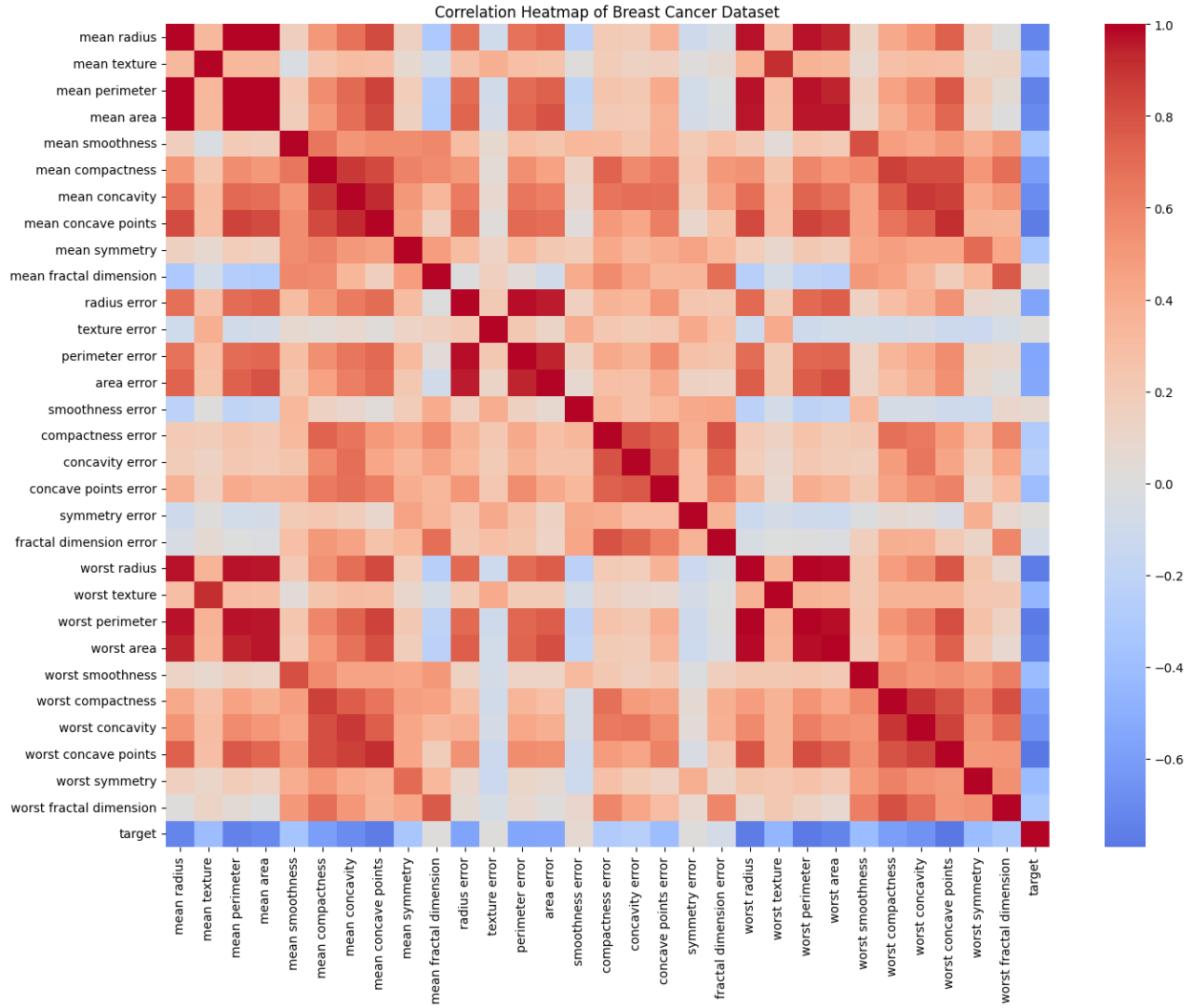
plt.title("Correlation Heatmap of Breast Cancer Dataset")
plt.show()

# Keep only the upper triangle using NumPy's upper-triangle indices
upper_values = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

strong_pairs = (
    upper_values.stack()
```

```
    .sort_values(ascending=False)
)

print("Top 10 positively correlated feature pairs:")
print(strong_pairs.head(10))
```




```
Top 10 positively correlated feature pairs:
mean radius       mean perimeter     0.997855
worst radius      worst perimeter    0.993708
mean radius       mean area          0.987357
mean perimeter    mean area          0.986507
worst radius      worst area         0.984015
worst perimeter   worst area         0.977578
radius error      perimeter error    0.972794
mean perimeter    worst perimeter    0.970387
mean radius       worst radius       0.969539
```

```
mean perimeter   worst radius       0.969476
dtype: float64
```

```
'''
5. Feature Engineering – Breast Cancer Dataset
a). Create a new feature called area_to_perimeter_ratio by dividing
mean area by `mean
  perimeter`. How does this new feature differ between benign and
malignant cases?

b). 2. Analyze the impact of `area_to_perimeter_ratio` on the
classification of cancer. Does this
new feature provide any additional insights or separability between
benign and malignant
cases?

How to interpret the results:
- If the two groups have noticeably different averages and
distributions, the feature may help distinguish benign from malignant
cases.
- If the distributions overlap substantially, this feature alone may
not classify cases accurately.
- The correlation with target indicates the direction and strength of
a linear relationship, but it does not prove that the feature improves
a classification model.

Remember: target = 0 means malignant and target = 1 means benign, so a
positive correlation indicates an association with benign cases.

'''
import matplotlib.pyplot as plt
# Create a new feature
df["area_to_perimeter_ratio"] = (
    df["mean area"] / df["mean perimeter"]
)

# Compare the ratio for benign and malignant cases
df["diagnosis"] = df["target"].map({
    0: "Malignant",
    1: "Benign"
})

ratio_summary = df.groupby("diagnosis")[
    "area_to_perimeter_ratio"
].agg(["mean", "median", "std"])

print(ratio_summary)
```

```python
df.boxplot(
    column="area_to_perimeter_ratio",
    by="diagnosis",
    figsize=(8, 5)
)

plt.title("Area-to-Perimeter Ratio by Diagnosis")
plt.suptitle("")
plt.xlabel("Diagnosis")
plt.ylabel("Area-to-Perimeter Ratio")
plt.show()


# Compare average ratio by diagnosis
print(df.groupby("diagnosis")[
    "area_to_perimeter_ratio"
].mean())

# Calculate correlation with target
correlation = df["area_to_perimeter_ratio"].corr(
    df["target"]
)

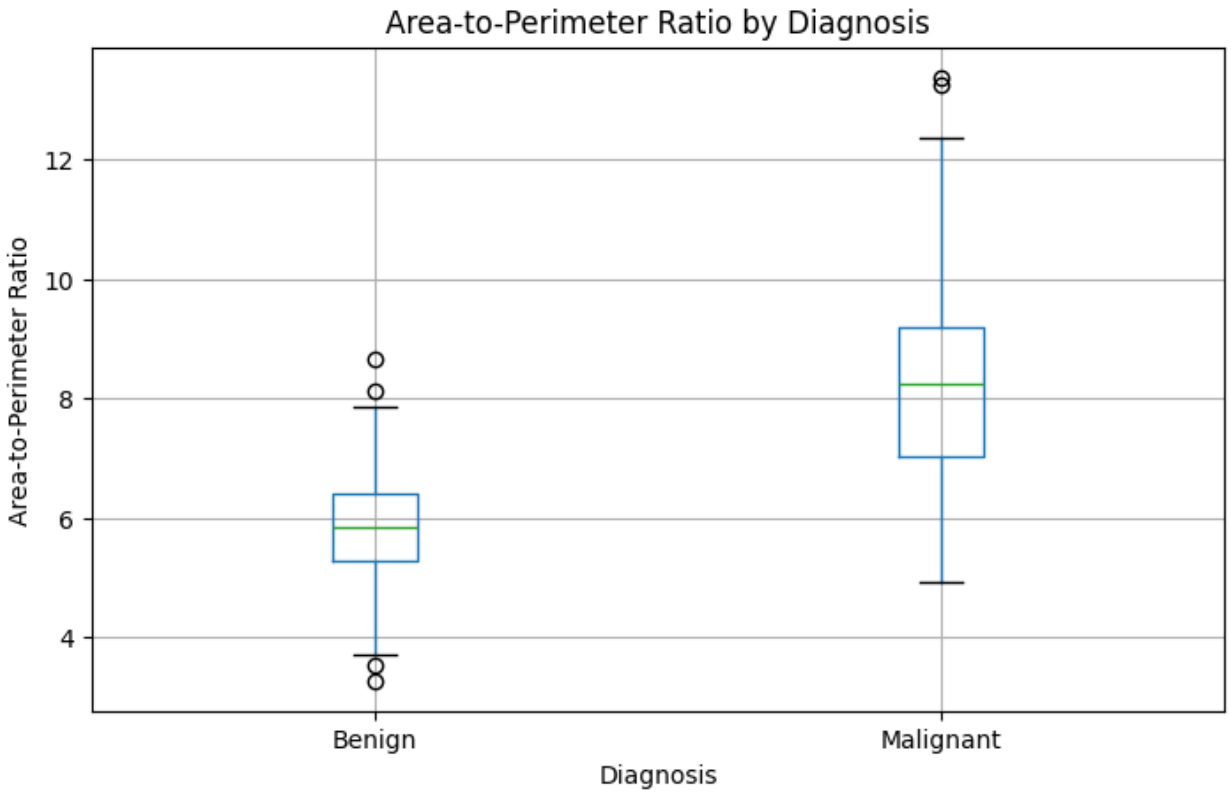
print("Correlation with target:", correlation)

# Plot distributions for both groups
plt.figure(figsize=(8, 5))

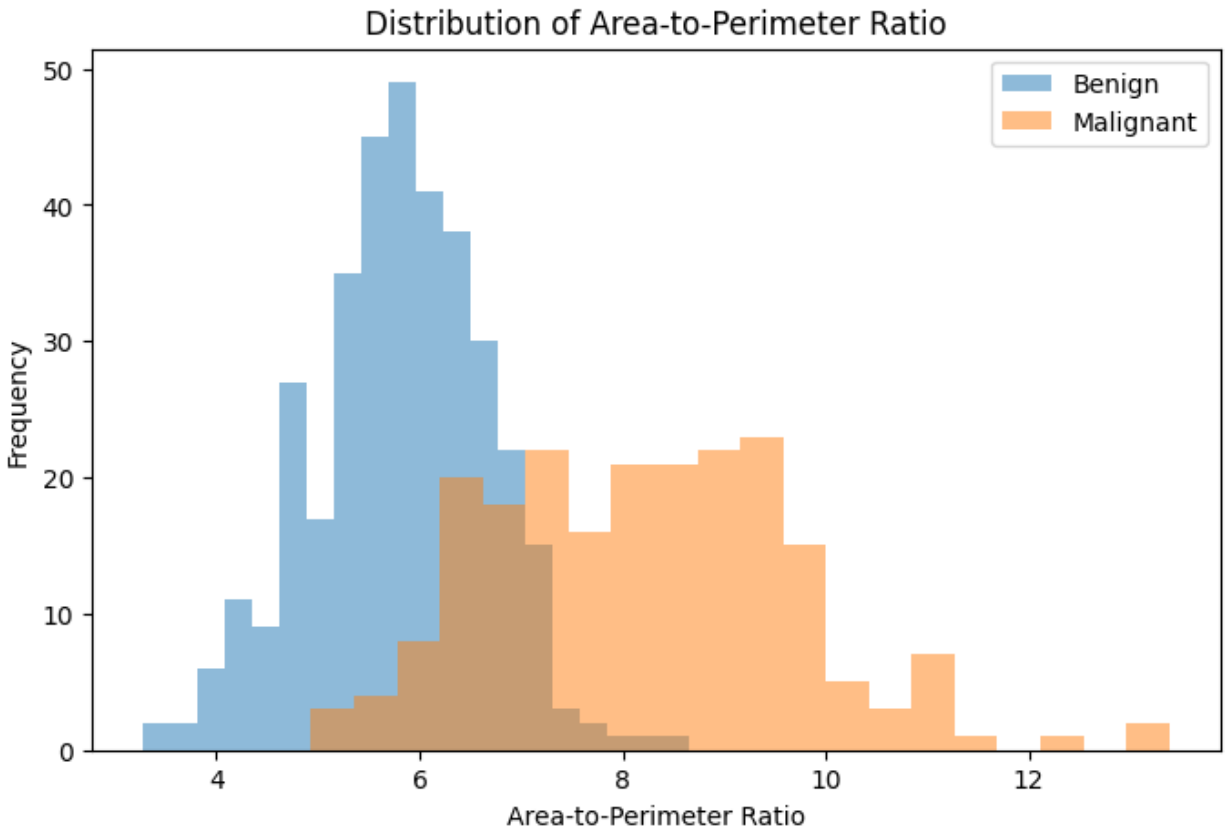
for diagnosis in ["Benign", "Malignant"]:
    values = df.loc[
        df["diagnosis"] == diagnosis,
        "area_to_perimeter_ratio"
    ]
    plt.hist(values, bins=20, alpha=0.5, label=diagnosis)

plt.xlabel("Area-to-Perimeter Ratio")
plt.ylabel("Frequency")
plt.title("Distribution of Area-to-Perimeter Ratio")
plt.legend()
plt.show()
```

```
                mean    median       std
diagnosis
Benign      5.800921  5.838105  0.855399
Malignant   8.207072  8.241443  1.495121
```




```
diagnosis
Benign       5.800921
Malignant    8.207072
Name: area_to_perimeter_ratio, dtype: float64
Correlation with target: -0.7159776513434114
```




```
'''
c). Are there other combinations of features that could be useful?
Create at least one more
derived feature and analyze its impact on the classification.
'''

# Create another derived feature
df["area_perimeter_shape_index"] = (
    df["mean area"] / (df["mean perimeter"] ** 2)
)

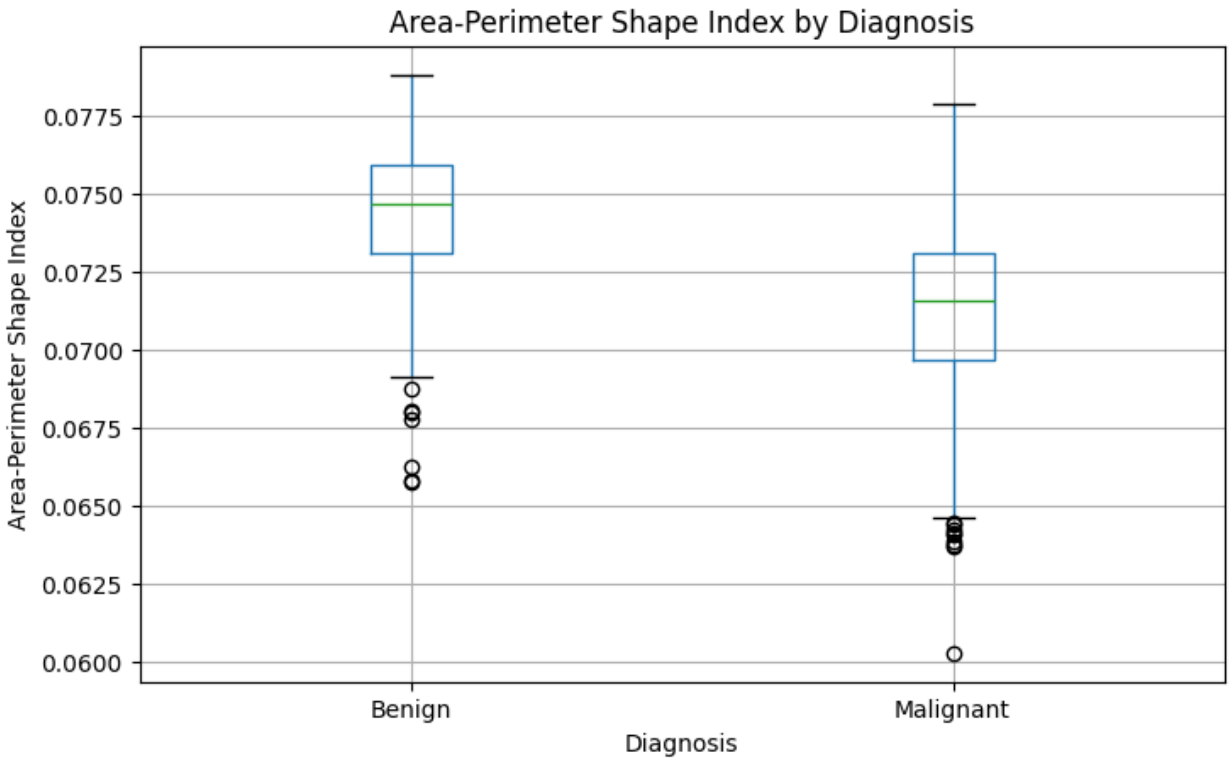
# Compare the feature between the two groups
print(
    df.groupby("diagnosis")[
        "area_perimeter_shape_index"
    ].agg(["mean", "median", "std"])
)

# Visualize the feature
df.boxplot(
    column="area_perimeter_shape_index",
    by="diagnosis",
    figsize=(8, 5)
)
```

```
plt.title("Area-Perimeter Shape Index by Diagnosis")
plt.suptitle("")
plt.xlabel("Diagnosis")
plt.ylabel("Area-Perimeter Shape Index")
plt.show()

# Check association with target
print(
    "Correlation with target:",
    df["area_perimeter_shape_index"].corr(df["target"])
)
```

diagnosis	mean	median	std
Benign	0.074370	0.074659	0.002151
Malignant	0.071274	0.071595	0.003050




```
Correlation with target: 0.5108137519890726
```

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load the breast cancer dataset
cancer = load_breast_cancer()
```

```python
# Create the DataFrame
df = pd.DataFrame(
    cancer.data,
    columns=cancer.feature_names
)

# Add target column: 0 = Malignant, 1 = Benign
df["target"] = cancer.target

# Add diagnosis labels
df["diagnosis"] = df["target"].map({
    0: "Malignant",
    1: "Benign"
})

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print(df.head())
```

```
Dataset loaded successfully!
Shape: (569, 32)
   mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
0        17.99         10.38          122.80     1001.0
0.11840
1        20.57         17.77          132.90     1326.0
0.08474
2        19.69         21.25          130.00     1203.0
0.10960
3        11.42         20.38           77.58      386.1
0.14250
4        20.29         14.34          135.10     1297.0
0.10030

   mean compactness  mean concavity  mean concave points  mean
symmetry  \
0           0.27760          0.3001              0.14710
0.2419
1           0.07864          0.0869              0.07017
0.1812
2           0.15990          0.1974              0.12790
0.2069
3           0.28390          0.2414              0.10520
0.2597
4           0.13280          0.1980              0.10430
0.1809

   mean fractal dimension  ...  worst perimeter  worst area  worst
smoothness  \
```

```
0                  0.07871  ...           184.60      2019.0
0.1622
1                  0.05667  ...           158.80      1956.0
0.1238
2                  0.05999  ...           152.50      1709.0
0.1444
3                  0.09744  ...            98.87       567.7
0.2098
4                  0.05883  ...           152.20      1575.0
0.1374

   worst compactness  worst concavity  worst concave points  worst
symmetry  \
0             0.6656           0.7119                0.2654
0.4601
1             0.1866           0.2416                0.1860
0.2750
2             0.4245           0.4504                0.2430
0.3613
3             0.8663           0.6869                0.2575
0.6638
4             0.2050           0.4000                0.1625
0.2364

   worst fractal dimension  target  diagnosis
0                  0.11890       0  Malignant
1                  0.08902       0  Malignant
2                  0.08758       0  Malignant
3                  0.17300       0  Malignant
4                  0.07678       0  Malignant

[5 rows x 32 columns]
'''
6. Conclusion
a) Summarize the key findings from your EDA. What were the most
significant patterns or
trends you observed?

Key findings:
- The dataset contains 569 cases, with 357 benign cases and 212
malignant cases.
- There are 30 original numerical measurement features and one target
variable.
- The original dataset contains no missing values.
- Malignant cases generally have higher values for measurements such
as mean radius, mean perimeter, and mean area.
- Several features are strongly correlated with one another,
especially measurements related to cell-nucleus size.
- Some overlap exists between benign and malignant cases, so a single
```

```
feature may not be enough to classify every case accurately
'''

# Number of benign and malignant cases
print("Diagnosis counts:")
print(df["diagnosis"].value_counts())

# Compare important features by diagnosis
features = [
    "mean radius",
    "mean perimeter",
    "mean area",
    "mean concavity",
    "mean concave points"
]

print("\nAverage feature values by diagnosis:")
print(df.groupby("diagnosis")[features].mean().round(2))

# Check missing values
print("\nTotal missing values:", df.isnull().sum().sum())
```

```
Diagnosis counts:
diagnosis
Benign       357
Malignant    212
Name: count, dtype: int64

Average feature values by diagnosis:
           mean radius  mean perimeter  mean area  mean concavity  \
diagnosis
Benign           12.15           78.08     462.79            0.05
Malignant        17.46          115.37     978.38            0.16

           mean concave points
diagnosis
Benign                    0.03
Malignant                 0.09

Total missing values: 0
```

```
'''
b). Based on your analysis, which features seem to be the most
important for distinguishing
between benign and malignant cases?

Based on the dataset's known patterns, the following features are
worth investigating first.
Feature                  Why it may be useful
mean concave          points    Measures the number and severity of
```

```
concave portions of the cell-nucleus boundary.
worst concave         points     Captures the most pronounced concave
boundary measurements.
worst perimeter       Represents a larger or more extreme perimeter
measurement.
worst area           Captures an extreme nucleus-area measurement.
mean concavity        Measures the severity of concave portions of the
boundary.
mean radius               Indicates nucleus size and often differs
between diagnosis groups.
These are candidate important features, not a definitive ranking from
your notebook.
'''

# Compare mean values for benign and malignant cases
important_features = [
    "mean concave points",
    "worst concave points",
    "worst perimeter",
    "worst area",
    "mean concavity",
    "mean radius"
]

print(
    df.groupby("diagnosis")[important_features]
    .mean()
    .round(3)
)
```

```
           mean concave points  worst concave points  worst perimeter
\
diagnosis

Benign                   0.026                 0.074           87.006

Malignant                0.088                 0.182          141.370


           worst area  mean concavity  mean radius
diagnosis
Benign        558.899           0.046       12.147
Malignant    1422.286           0.161       17.463
```

```
'''
c). What recommendations would you make for further analysis or model
building using this
dataset? Are there any areas where more data or different features
would be beneficial?
```

```
1. Feature selection: Identify the most informative features and
remove redundant ones where appropriate,
since many measurements are highly correlated.

2. Build classification models: Try Logistic Regression, Decision
Tree, Random Forest, and Support Vector Machine (SVM).

3. Evaluate model performance: Use a train-test split and cross-
validation. Evaluate precision, recall, F1-score, ROC-AUC,
and the confusion matrix. Pay particular attention to recall for
malignant cases.

4. Scale numerical features: Standardize features for models such as
Logistic Regression and SVM. Tree-based models generally
do not require feature scaling.

5. Evaluate derived features: Compare model performance with and
without area_to_perimeter_ratio and other engineered features
 to determine whether they add useful predictive information.

6. Explore additional data: If available, investigate additional
patient and clinical information, imaging measurements, or
longitudinal data.
 Ensure that any additional data is reliable, appropriately consented,
and representative.

7. Avoid data leakage: Perform feature selection and preprocessing
using only the training data within each cross-validation fold.
'''
```